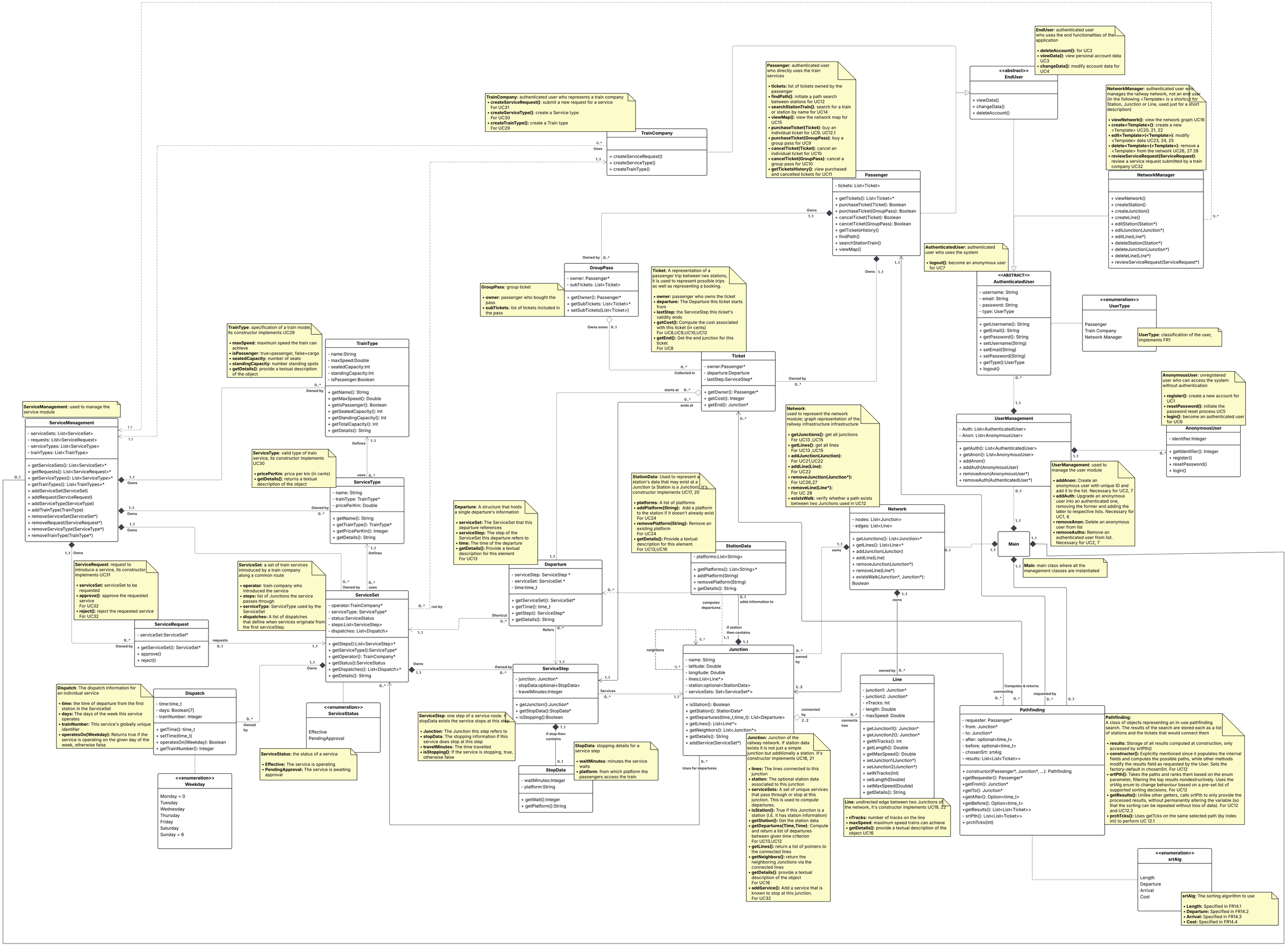
TrainCompany: authenticated user who represents a train company
• createServiceRequest(): submit a new request for a service For UC31
• createServiceType(): create a Service type For UC30
• createTrainType(): create a Train type For UC29
TrainCompany
+ createServiceRequest()
+ createServiceType()
+ createTrainType()
Uses
Passenger: authenticated user who directly uses the train services
• tickets: list of tickets owned by the passenger
• findPath(): initiate a path search between stations for UC12
• searchStationTrain(): search for a train or station by name for UC14
• viewMap(): view the network map for UC15
• purchaseTicket(Ticket): buy an individual ticket for UC9, UC12.1
• purchaseTicket(GroupPass): buy a group pass for UC9
• cancelTicket(Ticket): cancel an individual ticket for UC10
• cancelTicket(GroupPass): cancel a group pass for UC10
• getTicketsHistory(): view purchased and cancelled tickets for UC11
Passenger
- tickets: List<Ticket>
+ getTickets(): List<Ticket>*
+ purchaseTicket(Ticket): Boolean
+ purchaseTicket(GroupPass): Boolean
+ cancelTicket(Ticket): Boolean
+ cancelTicket(GroupPass): Boolean
+ getTicketsHistory()
+ findPath()
+ searchStationTrain()
+ viewMap()
Owns
EndUser: authenticated user who uses the end functionalities of the application
• deleteAccount(): for UC2
• viewData(): view personal account data UC3
• changeData(): modify account data for UC4
<<abstract>>
EndUser
+ viewData()
+ changeData()
+ deleteAccount()
NetworkManager: authenticated user who manages the railway network, not an end user (in the following <Template> is a shortcut for Station, Junction or Line, used just for a short description)
• viewNetwork(): view the network graph UC16
• create<Template>(): create a new <Template> UC20, 21, 22
• edit<Template>(<Template>): modify <Template> data UC23, 24, 25
• delete<Template>(<Template>): remove a <Template> from the network UC26, 27 28
• reviewServiceRequest(ServiceRequest): review a service request submitted by a train company UC32
NetworkManager
+ viewNetwork()
+ createStation()
+ createJunction()
+ createLine()
+ editStation(Station*)
+ editJuncction(Junction*)
+ editLine(Line*)
+ deleteStation(Station*)
+ deleteJunction(Junction*)
+ deleteLine(Line*)
+ reviewServiceRequest(ServiceRequest*)
AuthenticatedUser: authenticated user who uses the system
• logout(): become an anonymous user for UC7
<<ABSTRACT>>
AuthenticatedUser
- username: String
- email: String
- password: String
- type: UserType
+ getUsername(): String
+ getEmail(): String
+ getPassword(): String
+ setUsername(String)
+ setEmail(String)
+ setPassword(String)
+ getType():UserType
+ logout()
<<enumeration>>
UserType
Passenger
Train Company
Network Manager
UserType: classification of the user, implements FR1
AnonymousUser: unregistered user who can access the system without authentication
• register(): create a new account for UC1
• resetPassword(): initiate the password reset process UC5
• login(): become an authenticated user for UC6
AnonymousUser
- identifier:Integer
+ getIdentifier(): Integer
+ register()
+ resetPassword()
+ login()
UserManagement
- Auth: List<AuthenticatedUser>
- Anon: List<AnonymousUser>
+ getAuth(): List<AuthenticatedUser>
+ getAnon(): List<AnonymousUser>
+ addAnon()
+ addAuth(AnonymousUser)
+ removeAnon(AnonymousUser*)
+ removeAuth(AuthenticatedUser*)
UserManagement: used to manage the user module
• addAnon: Create an anonymous user with unique ID and add it to the list. Necessary for UC2, 7
• addAuth: Upgrade an anonymous user into an authenticated one, removing the former and adding the latter to respective lists. Necessary for UC1, 6
• removeAnon: Delete an anonymous user from list
• removeAutho: Remove an authenticated user from list. Necessary for UC2, 7
Owned by
GroupPass
- owner: Passenger*
- subTickets: List<Ticket>
+ getOwner(): Passenger*
+ getSubTickets: List<Ticket>*
+ setSubTickets(List<Ticket>)
GroupPass: group ticket
• owner: passenger who bought the pass
• subTickets: list of tickets included in the pass
Owns some
Ticket: A representation of a passenger trip between two stations, it is used to represent possible trips as well as representing a booking.
• owner: passenger who owns the ticket
• departure: The Departure this ticket starts from
• lastStep: the ServiceStep this ticket's validity ends
• getCost(): Compute the cost associated with this ticket (in cents) For UC8,UC9,UC10,UC12
• getEnd(): Get the end junction for this ticket For UC8
Ticket
- owner:Passenger*
- departure:Departure
- lastStep:ServiceStep*
+ getOwner(): Passenger*
+ getCost(): Integer
+ getEnd(): Junction*
Collected in
starts at
ends at
Owned by
TrainType: specification of a train model, its constructor implements UC29
• maxSpeed: maximum speed the train can achieve
• isPassenger: true=passenger, false=cargo
• seatedCapacity: number of seats
• standingCapacity: number standing spots
• getDetails(): provide a textual description of the object
TrainType
- name:String
- maxSpeed:Double
- seatedCapacity:Int
- standingCapacity:Int
- isPassenger:Boolean
+ getName(): String
+ getMaxSpeed(): Double
+ getIsPassenger(): Boolean
+ getSeatedCapacity(): Int
+ getStandingCapacity(): Int
+ getTotalCapacity(): Int
+ getDetails(): String
Defines
ServiceManagement: used to manage the service module
ServiceManagement
- serviceSets: List<ServiceSet>
- requests: List<ServiceRequest>
- serviceTypes: List<ServiceType>
- trainTypes: List<TrainType>
+ getServiceSets(): List<ServiceSet>*
+ getRequests(): List<ServiceRequest>*
+ getServiceTypes(): List<ServiceType>*
+ getTrainTypes(): List<TrainTypes>*
+ addServiceSet(ServiceSet)
+ addRequest(ServiceRequest)
+ addServiceType(ServiceType)
+ addTrainType(TrainType)
+ removeServiceSet(ServiceSet*)
+ removeRequest(ServiceRequest*)
+ removeServiceType(ServiceType*)
+ removeTrainType(TrainType*)
Owns
ServiceType: valid type of train service, its constructor implements UC30
• pricePerKm: price per km (in cents)
• getDetails(): returns a textual description of the object
ServiceType
- name: String
- trainType: TrainType*
- pricePerKm: Double
+ getName(): String
+ getTrainType(): TrainType*
+ getPricePerKm(): Integer
+ getDetails(): String
uses
Network: used to represent the network module; graph representation of the railway infrastructure infrastructure
• getJunctions(): get all junctions For UC13 ,UC15
• getLines(): get all lines For UC13 ,UC15
• addJunction(Junction): For UC21,UC22
• addLine(Line): For UC22
• removeJunction(Junction*): For UC26,27
• removeLine(Line*): For UC 28
• existsWalk: verify whether a path exists between two Junctions used in UC12
Network
- nodes: List<Junction>
- edges: List<Line>
+ getJunctions(): List<Junction>*
+ getLines(): List<Line>*
+ addJunction(Junction)
+ addLine(Line)
+ removeJunction(Junction*)
+ removeLine(Line*)
+ existsWalk(Junction*, Junction*): Boolean
owns
Main
Main: main class where all the management classes are instantiated
StationData: Used to represent a station's data that may exist at a Junction (a Station is a Junction). It's constructor implements UC17, 20
• platforms: A list of platforms
• addPlatform(String): Add a platform to the station if it doesn't already exist For UC24
• removePlatform(String): Remove an existing platform For UC24
• getDetails(): Provide a textual description for this element. For UC13,UC16
StationData
- platforms:List<String>
+ getPlatforms(): List<String>*
+ addPlatform(String)
+ removePlatform(String)
+ getDetails(): String
Departure: A structure that holds a single departure's information
• serviceSet: The ServiceSet that this departure references
• serviceStep: The step of the ServiceSet this departure refers to
• time: The time of the departure
• getDetails(): Provide a textual description for this element For UC13
Departure
- serviceStep: ServiceStep *
- serviceSet: ServiceSet *
- time:time_t
+ getServiceSet(): ServiceSet*
+ getTime(): time_t
+ getStep(): ServiceStep*
+ getDetails(): String
Refers
computes departures
adds information to
if station then contains
ServiceRequest: request to introduce a service, its constructor implements UC31
• serviceSet: serviceSet to be requested
• approve(): approve the requested service For UC32
• reject(): reject the requested service For UC32
ServiceRequest
- serviceSet:ServiceSet*
+ getServiceSet(): ServiceSet*
+ approve()
+ reject()
requests
ServiceSet: a set of train services introduced by a train company along a common route
• operator: train company who introduced the service
• steps: list of Junctions the service passes through
• serviceType: ServiceType used by the ServiceSet
• dispatches: A list of dispatches that define when services originate from the first serviceStep.
ServiceSet
- operator:TrainCompany*
- serviceType: ServiceType*
- status:ServiceStatus
- steps:List<ServiceStep>
- dispatches: List<Dispatch>
+ getSteps():List<ServiceStep>*
+ getServiceType():ServiceType*
+ getOperator(): TrainCompany*
+ getStatus():ServiceStatus
+ getDispatches(): List<Dispatch>*
+ getDetails(): String
run by
Shortcut
Owned by
uses
Junction: Junction of the railway network. If station data exists it is not just a simple junction but additionally a station. It's constructor implements UC18, 21
• lines: The lines connected to this junction
• station: The optional station data associated to this junction
• serviceSets: A set of unique services that pass through or stop at this junction. This is used to compute departures.
• isStation(): True if this Junction is a station (I.E. It has station information)
• getStation(): Get the station data
• getDepartures(Time,Time): Compute and return a list of departures between given time criterion For UC13,UC12
• getLines(): return a list of pointers to the connected lines
• getNeighbors(): return the neighboring Junctions via the connected lines
• getDetails(): provide a textual description of the object For UC16
• addService(): Add a service that is known to stop at this junction. For UC32
Junction
- name: String
- latitude: Double
- longitude: Double
- lines:List<Line*>
- station:optional<StationData>
- serviceSets: Set<ServiceSet*>
+ isStation(): Boolean
+ getStation(): StationData*
+ getDepartures(time_t,time_t): List<Departure>
+ getLines(): List<Line*>
+ getNeighbors(): List<Junction*>
+ getDetails(): String
+ addService(ServiceSet*)
neighbors
owned by
connected by
connects two
Uses for departures
Line: undirected edge between two Junctions of the network, it's constructor implements UC19, 22
• nTracks: number of tracks on the line
• maxSpeed: maximum speed trains can achieve
• getDetails(): provide a textual description of the object UC16
Line
- junction1: Junction*
- junction2: Junction*
- nTracks: Int
- length: Double
- maxSpeed: Double
+ getJunction1(): Junction*
+ getJunction2(): Junction*
+ getNTracks(): Int
+ getLength(): Double
+ getMaxSpeed(): Double
+ setJunction1(Junction*)
+ setJunction2(Junction*)
+ setNTracks(Int)
+ setLength(Double)
+ setMaxSpeed(Double)
+ getDetails(): String
Pathfinding: A class of objects representing an in-use pathfinding search. The results of the search are stored each as a list of stations and the tickets that would connect them
• results: Storage of all results computed at construction, only accessed by srtPth()
• constructor(): Explicitly mentioned since it populates the internal fields and computes the possible paths, while other methods modify the results field as requested by the User. Sets the factory-default in chosenSrt. For UC12
• srtPth(): Takes the paths and ranks them based on the enum parameter, filtering the top results nondestructively. Uses the srtAlg enum to change behaviour based on a pre-set list of supported sorting decisions. For UC12
• getResults(): Unlike other getters, calls srtPth to only provide the processed results, without permanently altering the variable (so that the sorting can be repeated without loss of data). For UC12 and UC12.2
• prchTcks(): Uses getTcks on the same selected path (by index int) to perform UC 12.1
Pathfinding
- requester: Passenger*
- from: Junction*
- to: Junction*
- after: optional<time_t>
- before: optional<time_t>
- chosenSrt: srtAlg
- results: List<List<Ticket>>
+ constructor(Passenger*, Junction*, ...): Pathfinding
+getRequester(): Passenger*
+getFrom(): Junction*
+getTo(): Junction*
+getAfter(): Option<time_t>
+getBefore(): Option<time_t>
+getResults(): List<List<Ticket>>
- srtPth(): List<List<Ticket>>
+ prchTcks(int)
Computes & returns
connecting
requested by
<<enumeration>>
srtAlg
Length
Departure
Arrival
Cost
srtAlg: The sorting algorithm to use
• Length: Specified in FR14.1
• Departure: Specified in FR14.2
• Arrival: Specified in FR14.3
• Cost: Specified in FR14.4
ServiceSet
- operator:TrainCompany*
- serviceType: ServiceType*
- status:ServiceStatus
- steps:List<ServiceStep>
- dispatches: List<Dispatch>
+ getSteps():List<ServiceStep>*
+ getServiceType():ServiceType*
+ getOperator(): TrainCompany*
+ getStatus():ServiceStatus
+ getDispatches(): List<Dispatch>*
+ getDetails(): String
Owns
Owned by
ServiceStep: one step of a service route. If stopData exists the service stops at this step
• Junction: The Junction this step refers to
• stopData: The stopping information if this service does stop at this step
• travelMinutes: The time travelled
• isStopping(): If the service is stopping, true, otherwise false
ServiceStep
- junction: Junction*
- stopData:optional<StopData>
- travelMinutes:Integer
+ getJunction():Junction*
+ getStopData():StopData*
+ isStopping():Boolean
Services
if stop then contains
StopData: stopping details for a service step
• waitMinutes: minutes the service waits
• platform: from which platform the passengers access the train
StopData
- waitMinutes:Integer
- platform:String
+ getWait():Integer
+ getPlatform():String
Dispatch: The dispatch information for an individual service
• time: the time of departure from the first station in the ServiceSet
• days: The days of the week this service operates
• trainNumber: This service's globally unique identifier
• operatesOn(Weekday): Returns true if the service is operating on the given day of the week, otherwise false
Dispatch
- time:time_t
- days: Boolean[7]
- trainNumber: Integer
+ getTime(): time_t
+ setTime(time_t)
+ operatesOn(Weekday): Boolean
+ getTrainNumber(): Integer
Owned by
<<enumeration>>
ServiceStatus
Effective
PendingApproval
ServiceStatus: the status of a service
• Effective: The service is operating
• PendingApproval: The service is awaiting approval
<<enumeration>>
Weekday
Monday = 0
Tuesday
Wednesday
Thursday
Friday
Saturday
Sunday = 6